

SYSTEM DESIGN.

↳ OOPS.

→ Procedural programming:-
Functions
Loops
Blocks (if-else, switches).

→ OOPS:-
Real-world modelling.
Data security.
Highly scalable and reusable algorithms.

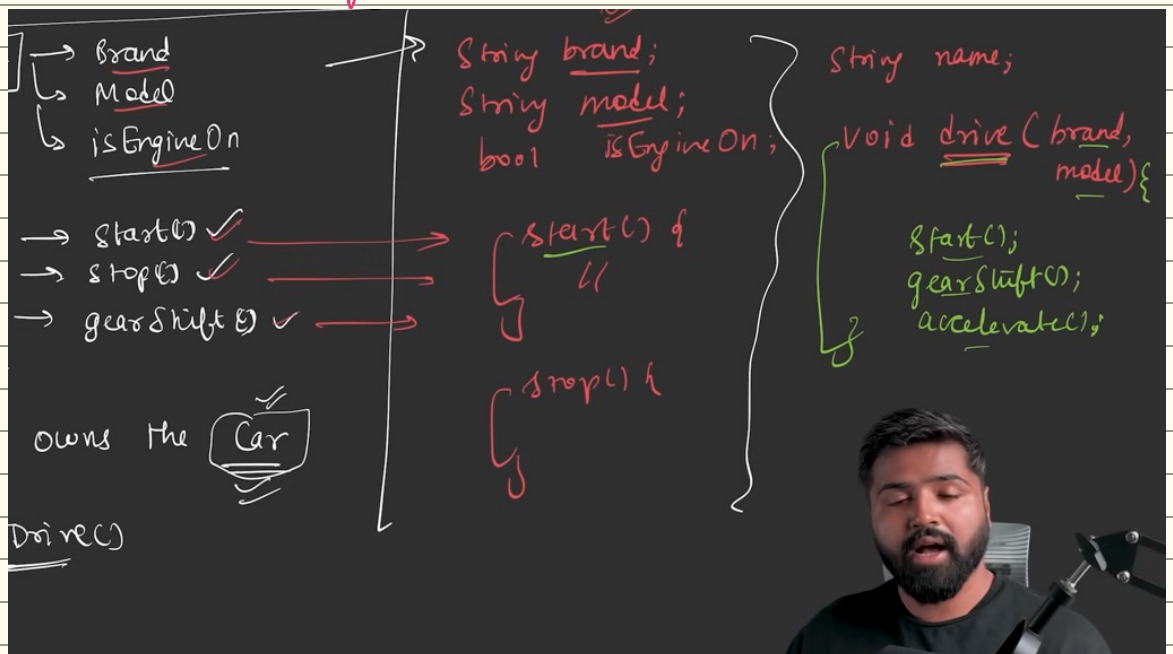
OBJECTS:-

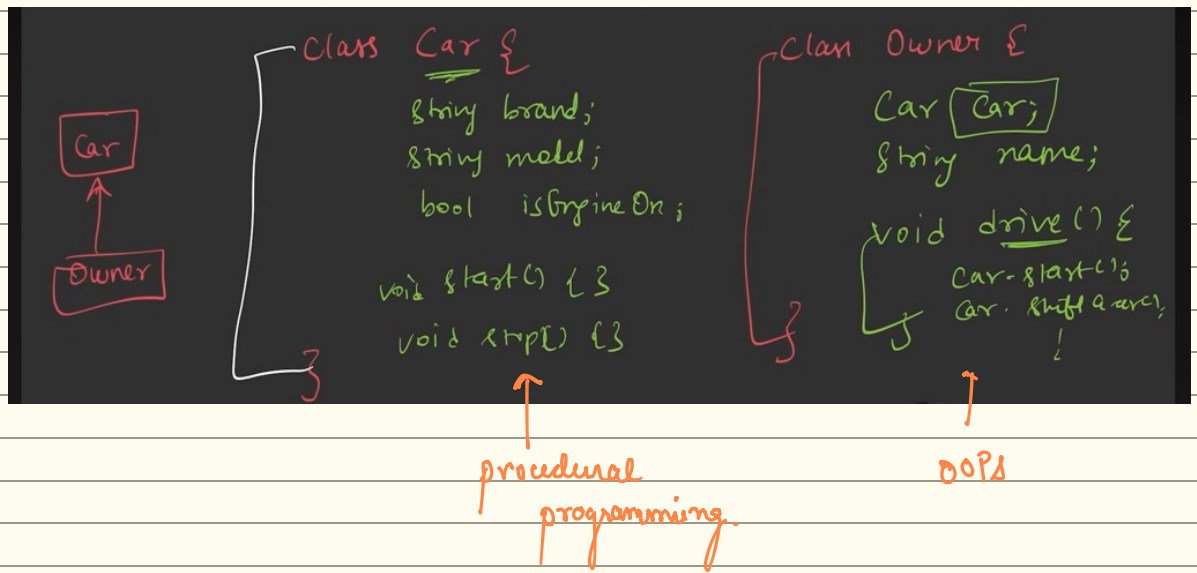
↳ Characteristics
↳ Behaviour.

CAR Characteristics
↳ Engine
↳ Brand
↳ Model
↳ Wheels.

Behaviour
↳ start()
↳ stop()
↳ gearShift()
↳ acceleration()
↳ brakes()

Ex:- Representing a car:-





→ **OOPS** :- OOPs (Object-Oriented Programming System) is a programming paradigm that organizes code into objects - real-world entities that contain data (attributes) and behaviors (methods).

It helps make code more modular, reusable, scalable, and easier to maintain.

→ **Pillars of OOPS** :-

- (i). Abstraction.
- (ii). Encapsulation.
- (iii). Inheritance.
- (iv). Polymorphism.

→ **Abstraction** :-

4. Abstraction — "Show only essential things"

Hiding complex implementation from the user and showing only the required functionality.

Achieved by:

- Abstract classes
- Interfaces

Example:

```

java
Copy code

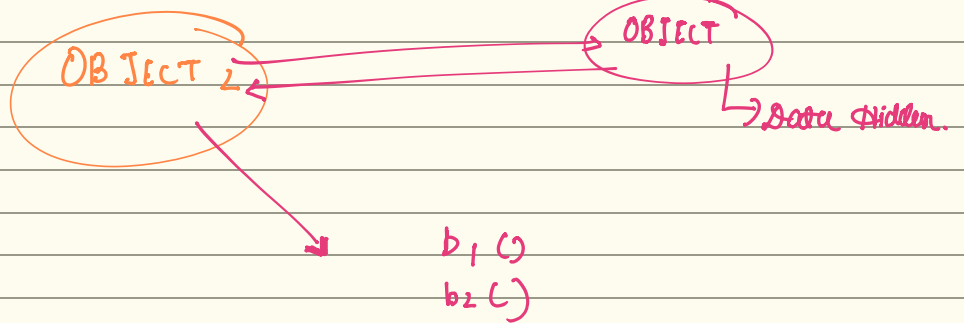
abstract class Car {
    abstract void start();
}

class Tesla extends Car {
    void start() { System.out.println("Starting Tesla"); }
}

```

Owner → Car

- ↳ accelerate - ()
- ↳ brakes - ()
- ↳ gearshift - ()



Abstraction hides unnecessary details from a client, and showcase only what is necessary.

```
interface Vehicle {
    void start(); // abstract method by default
    void stop();
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }

    public void stop() {
        System.out.println("Car stops");
    }
}

class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v.stop();
    }
}
```

→ Encapsulation:- Data + Methods bundled together.

Encapsulation means wrapping data (variables) and methods (functions) inside a single unit (class).

Key ideas:-

- Hides internal details.
- Expose only what is necessary.

→ This is denoted via a class:-

Car → Characters
→ Behaviour.

class car {

// Variables → Characters.

// Methods → Behaviour

}

Hence, we use access modifiers:-

→ public
→ private
→ protected

```
class Student {  
    // Private data → cannot be accessed directly  
    private String name;  
    private int age;  
  
    // Public setter methods → allow controlled modification  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setAge(int age) {  
        if (age > 0) { // validation  
            this.age = age;  
        } else {  
            System.out.println("Invalid age");  
        }  
    }  
  
    // Public getter methods → allow controlled access  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        // Setting values through setters  
        s.setName("Pratyush");  
        s.setAge(21);  
  
        // Accessing values through getters  
        System.out.println("Name: " + s.getName());  
        System.out.println("Age: " + s.getAge());  
    }  
}
```